MINOR PROJECT – 2 Aashvika Dubey

Zero Trust Architecture

for Enterprise Security

Objective: Explore the concept of Zero Trust and implement it in corporate environments using MFA, RBAC, micro-segmentation, and IAM systems.

Step 1: Understand Zero Trust Principles

1.1 What Is Zero Trust?

Zero Trust is a cybersecurity architecture model built on a single foundational principle: Never Trust, Always Verify. Unlike traditional network security models that assumed everything inside the corporate network was safe (the 'castle and moat' model), Zero Trust assumes that threats exist both inside and outside the network at all times.

The Zero Trust model was formally introduced by analyst John Kindervag at Forrester Research in 2010 and has since been adopted as the recommended framework by the US federal government, NIST, and most Fortune 500 companies. Its relevance has grown dramatically in the era of cloud computing, remote work, and mobile devices, where the traditional network perimeter no longer exists.

1.2 The Three Core Pillars of Zero Trust

Pillar 1: Verify Identity Explicitly

Every access request — regardless of whether it comes from inside the office network or from a remote employee — must be authenticated and authorized. This means:

- Strong multi-factor authentication (MFA) for every login, every time
- Continuous validation: a user authenticated at 9 AM may need to re-authenticate if behavior changes or a new sensitive resource is accessed
- Device identity: not just who the user is, but whether the device they are using is managed, up-to-date, and compliant
- Context-aware access: location, time of day, and risk score are factors in granting access

Pillar 2: Use Least Privilege Access

Users, applications, and systems should be granted only the minimum level of access required to perform their specific function — and no more. This is formalized as the Principle of Least Privilege (PoLP):

- A customer service agent can view order history but cannot access payment processing systems
- An application server can read from a database but cannot write to backup storage
- Privileged access (admin rights) is time-limited: granted for a specific task and automatically revoked after
- Just-In-Time (JIT) access: sensitive system access is provisioned on-demand and expires automatically

Pillar 3: Assume Breach

Zero Trust requires designing the entire system as if an attacker is already inside the network. This changes how security is engineered:

- Network segmentation: divide the network into isolated zones so a breach in one zone cannot spread
- Encrypt all traffic: even internal east-west traffic between servers is encrypted, so an attacker who gains network access cannot read it
- Comprehensive logging: every access request, file read, login attempt, and network connection is logged for forensic analysis
- Continuous monitoring: anomaly detection tools watch for unusual behavior patterns in real time

1.3 Zero Trust vs. Traditional Security

Dimension	Traditional Security	Zero Trust Security
Trust Model	Trust everything inside the perimeter firewall	Trust nothing — verify everything on every access
Network Perimeter	Hard outer perimeter; flat internal network	No perimeter; micro-segmented network zones
User Access	Authenticate once at login; trusted for the session	Continuous verification; context-aware access decisions
Remote Workers	VPN tunnels everything through the firewall	Identity-based access; split tunneling; ZTNA
Breach Assumption	Breach is an exception to be prevented	Breach is assumed; limit blast radius by design
Monitoring	Perimeter-focused; internal traffic rarely logged	All traffic logged; AI anomaly detection applied

Step 2: Define Authentication Models

2.1 Multi-Factor Authentication (MFA)

Multi-Factor Authentication requires users to prove their identity using two or more independent verification factors from different categories. Even if one factor (such as a password) is compromised, an attacker cannot gain access without the additional factors.

The Three Authentication Factors

Factor Type	What It Means	Examples
Something You Know	Knowledge-based factor	Password, PIN, security questions, passphrase
Something You Have	Possession-based factor	Authenticator app (Google Auth, Microsoft Auth), hardware token (YubiKey), SMS OTP, smart card
Something You Are	Biometric factor	Fingerprint, facial recognition, iris scan, voice recognition

MFA Implementation for the Project

For your student lab simulation, you can implement MFA using:

- Google Authenticator or Microsoft Authenticator (free TOTP apps) as the second factor
- Keycloak (open-source IAM) natively supports TOTP-based MFA — enable it in the realm settings
- FreeOTP: a free, open-source authenticator app for Android/iOS that works with TOTP standards
- Test by attempting login with correct password but wrong OTP — system should deny access

2.2 Role-Based Access Control (RBAC)

RBAC is an access control model where permissions are assigned to roles, and users are assigned to roles based on their job function. This decouples permission management from individual user management, making it scalable and auditable.

RBAC Structure

- Users are assigned to one or more Roles (e.g., User: Alice → Role: Finance_Manager)
- Roles are granted Permissions (e.g., Role: Finance_Manager → Permission: Read_Payroll, Write_Invoices)
- Permissions map to Resources (e.g., Permission: Read_Payroll → Resource: /data/payroll/*.xlsx)

Sample RBAC Policy for Enterprise Lab Simulation

Role	Department	Allowed Resources	Denied Resources
IT_Admin	IT	All systems; server configs; IAM management	HR personal files (separation of duties)
HR_Manager	Human Resources	Employee records; payroll system; onboarding docs	Financial systems; production servers
Finance_Analyst	Finance	Invoice system; financial reports; budget sheets	HR records; IT infrastructure; source code repos
Developer	Engineering	Source code repos; dev/staging environments; CI/CD pipeline	Production customer data; HR systems; financial records
Guest	Visitor / Contractor	Internet access only; designated shared folder	All internal systems; file shares; admin panels

2.3 How MFA and RBAC Work Together in Zero Trust

MFA and RBAC are complementary controls that together form the access control layer of Zero Trust:

1. User attempts to access a protected resource
2. System triggers MFA: user enters password (factor 1) + OTP from authenticator app (factor 2)
3. Identity is verified — authentication is successful
4. System checks the user's assigned role(s) in the RBAC policy
5. System checks whether that role has permission to access the requested resource
6. If both checks pass: access is granted and the event is logged
7. If either check fails: access is denied, the event is logged, and an alert may be triggered

Step 3: Design the Zero Trust Framework

3.1 The Zero Trust Architecture Layers

A complete Zero Trust framework for enterprise security consists of five interconnected layers. Each layer enforces security independently, so that even if one layer is bypassed, the others continue to protect the organization.

Layer 1: Identity and Access Management (IAM)

The foundation of Zero Trust. Every human user and machine service must have a verified digital identity before accessing any resource.

- Centralized identity directory (e.g., Microsoft Active Directory, Keycloak, Okta)
- MFA enforced for all accounts — no exceptions for senior staff or IT administrators
- Single Sign-On (SSO) with federated identity for cloud applications
- Privileged Identity Management (PIM): elevated access is temporary and audited

Layer 2: Device Trust and Endpoint Security

Zero Trust extends verification to the device being used, not just the user. A valid user on a compromised device is still a security risk.

- Mobile Device Management (MDM): company devices must be enrolled and compliant before access is granted
- Device health checks: OS must be patched, antivirus must be active, disk encryption must be enabled
- Personal devices (BYOD): access limited to non-sensitive resources unless enrolled in MDM
- Certificate-based authentication for devices, not just users

Layer 3: Network Micro-Segmentation

Instead of one flat network where any device can reach any other, Zero Trust divides the network into isolated segments (microsegments). Each segment has strict firewall rules governing what traffic is permitted.

- Separate VLANs for HR, Finance, Engineering, Guest, and IoT devices
- East-West firewall policies: traffic between internal segments is blocked by default and must be explicitly allowed
- Software-Defined Networking (SDN): network policies are defined in software and applied programmatically
- Zero Trust Network Access (ZTNA): replaces VPN for remote workers — access is granted per-application, not per-network

Layer 4: Application Security

Applications themselves enforce access controls rather than relying on network controls alone.

- Application-level authentication: each application validates the identity token before serving any request
- API gateways with rate limiting and token validation (OAuth 2.0, JWT)
- Web Application Firewall (WAF) for externally facing applications
- Secrets management: no hardcoded passwords in application code; use tools like HashiCorp Vault

Layer 5: Data Security and Encryption

The final layer protects data regardless of where it resides or how it is transmitted.

- Encryption at rest: all stored data — databases, file servers, cloud storage — uses AES-256 encryption
- Encryption in transit: all network communication uses TLS 1.3; no unencrypted protocols (HTTP, FTP, Telnet)
- Data classification: categorize data as Public, Internal, Confidential, or Restricted — apply controls accordingly
- Data Loss Prevention (DLP): monitor and block unauthorized exfiltration attempts

3.2 Zero Trust Architecture Diagram (Textual Description)

The following describes the logical flow through your Zero Trust architecture, which you should draw as a diagram for your project submission:

- External Users and Remote Workers connect via ZTNA (not traditional VPN)
- All access requests pass through the Policy Decision Point (PDP): the IAM system checks identity, MFA status, device compliance, and RBAC permissions
- The Policy Enforcement Point (PEP) sits in front of each resource segment and enforces the PDP's decision
- The internal network is divided into micro-segments: [HR Zone] [Finance Zone] [Engineering Zone] [Production Zone] [Guest Zone]
- A Security Information and Event Management (SIEM) system receives logs from all layers for centralized monitoring

Step 4: Simulate Using VMware and Docker

4.1 Lab Setup Overview

You will build a virtual enterprise network using virtualization tools to simulate the Zero Trust architecture. This gives you a real environment to configure, test, and demonstrate the framework without physical hardware.

4.2 VMware / VirtualBox Lab Setup

Required Virtual Machines

VM Name	OS	Role	Network Segment
IAM-Server	Ubuntu 22.04	Keycloak IAM, LDAP directory	Management VLAN (10.0.0.x)
HR-Workstation	Windows 10/11	Simulates HR department user	HR VLAN (10.0.1.x)
Finance-Workstation	Windows 10/11	Simulates Finance department user	Finance VLAN (10.0.2.x)
File-Server	Ubuntu 22.04	Hosts sensitive data (Samba shares)	Data VLAN (10.0.3.x)
Attacker-VM	Kali Linux	Simulate insider/external threats	Guest VLAN (10.0.99.x)

Network Configuration Steps

8. In VMware/VirtualBox, create 5 host-only networks corresponding to the VLANs above
9. Assign each VM to its designated network adapter — only the IAM-Server needs access to multiple VLANs (acting as the Policy Decision Point)
10. Configure firewall rules using UFW (Ubuntu) or Windows Firewall to block cross-VLAN traffic by default
11. Add explicit allow rules: HR-Workstation can reach File-Server on port 445 (Samba) only after IAM authentication
12. Verify segmentation: from HR-Workstation, attempt to ping Finance-Workstation — this should fail

4.3 Docker Lab Setup

Use Docker to simulate isolated application services that enforce their own access controls:

Docker Services to Deploy

- Keycloak: run as a Docker container for the IAM/SSO server
- Nginx reverse proxy: sit in front of application containers and validate JWT tokens before forwarding requests
- A sample web application (any simple Flask or Node.js app): requires a valid JWT from Keycloak to serve content

- Prometheus + Grafana: monitoring stack to visualize access logs and alert on anomalies

Step-by-Step Docker Commands

```
# Pull and run Keycloak
docker run -d -p 8080:8080 -e KEYCLOAK_ADMIN=admin -e
KEYCLOAK_ADMIN_PASSWORD=admin quay.io/keycloak/keycloak:latest start-dev
# Create isolated Docker networks for each simulated department
docker network create hr-network
docker network create finance-network
docker network create guest-network
```

Deploy each department's application container on its own network. Configure Nginx to act as the Policy Enforcement Point, requiring a valid bearer token (issued by Keycloak after MFA) for every request.

Step 5: Implement IAM Access Control Policies

5.1 Setting Up Keycloak IAM

Keycloak is an enterprise-grade, open-source IAM system that provides SSO, MFA, RBAC, and OAuth 2.0/OIDC out of the box. It is widely used in production environments (Red Hat, JBoss) and is perfect for your simulation.

Keycloak Configuration Steps

13. Access the Keycloak admin console at http://localhost:8080/admin — log in with the admin credentials set during Docker setup
14. Create a new Realm named 'enterprise-zt' (a realm is an isolated tenant/organization in Keycloak)
15. Create Roles matching your RBAC policy: IT_Admin, HR_Manager, Finance_Analyst, Developer, Guest
16. Create Users and assign each user to their appropriate role (e.g., user 'alice' → role 'HR_Manager')
17. Enable OTP (One-Time Password) MFA in the Authentication flows: go to Authentication → Flows → Browser → add OTP Form as Required
18. Create a Client (application) that will use Keycloak for authentication — set the redirect URI to your application's URL
19. Test by logging in as each user and verifying that the TOTP prompt appears (scan the QR code with Google Authenticator)

5.2 Configuring RBAC Policies in Keycloak

Fine-Grained Authorization

Keycloak's Authorization Services allow you to define resource-level permissions directly in the IAM system:

- Create Resources: define the protected resources (e.g., 'HR_Database', 'Finance_Reports', 'Source_Code_Repository')
- Create Policies: define who can access each resource based on role (Role-Based Policy: only HR_Manager can access HR_Database)
- Create Permissions: link resources and policies together
- Test: log in as Finance_Analyst and attempt to access an HR resource — Keycloak should return a 403 Forbidden response

5.3 Implementing Policies on Linux (Alternative Approach)

If using a simpler Linux-based simulation without Keycloak, you can implement RBAC using Linux user groups and file system permissions:

```
# Create department groups

sudo groupadd hr_dept

sudo groupadd finance_dept

sudo groupadd engineering_dept
```

```
# Create users and assign to groups

sudo useradd -m alice && sudo usermod -aG hr_dept alice

sudo useradd -m bob && sudo usermod -aG finance_dept bob

# Create directories with group-restricted permissions

sudo mkdir /data/hr && sudo chown root:hr_dept /data/hr && sudo chmod 770 /
data/hr

sudo mkdir /data/finance && sudo chown root:finance_dept /data/finance &&
sudo chmod 770 /data/finance
```

Verify: log in as 'alice' (HR user) and attempt to list files in /data/finance — should receive 'Permission denied'.

5.4 Access Control Policy Summary Table

Policy Name	Subject (Who)	Resource (What)	Effect
HR Data Access	HR_Manager role	/data/hr/*, HR application	**ALLOW**
HR Data Block	All other roles	/data/hr/*, HR application	**DENY**
Finance Data Access	Finance_Analyst role	/data/finance/*, Finance app	**ALLOW**
Finance Data Block	All other roles	/data/finance/*, Finance app	**DENY**
Admin Full Access	IT_Admin role	All systems, IAM console	**ALLOW**
Guest Isolation	Guest role	Internet only; no internal systems	**DENY ALL INTERNAL**

Step 6: Test Security Effectiveness

6.1 Testing Methodology

Security testing validates that the Zero Trust controls you have implemented actually work as intended. You will conduct two categories of tests — simulating insider threats (a legitimate user trying to exceed their privileges) and external threats (an unauthorized actor trying to gain access).

6.2 Insider Threat Simulation Tests

Test 1: Privilege Escalation Attempt

- Login as 'bob' (Finance_Analyst role)
- Attempt to access /data/hr/ or the HR application
- Expected result: Access Denied (403 Forbidden or 'Permission denied' in Linux)
- Document: screenshot of the denial, check the audit log to confirm the attempt was recorded

Test 2: Lateral Movement Within Network

- From the Finance-Workstation VM, attempt to ping or connect to the HR-Workstation VM
- Expected result: ICMP timeout or connection refused — micro-segmentation prevents cross-VLAN communication
- Document: Wireshark capture showing the blocked traffic

Test 3: Data Exfiltration Attempt

- As a low-privilege user, attempt to copy a large number of files to an external location
- If DLP rules are configured, this should trigger an alert
- Expected result: transfer blocked or alert generated in the monitoring dashboard

Test 4: MFA Bypass Attempt

- Obtain a valid username and password for a test account
- Attempt to log in without the MFA token (close the MFA prompt)
- Expected result: Login fails — system requires valid OTP before granting access

6.3 External Threat Simulation Tests

Test 5: Network Port Scanning (from Attacker VM)

- From the Kali Linux Attacker VM (on the Guest VLAN), run an Nmap scan against the internal VLANs
- Command: nmap -sS 10.0.1.0/24 (targeting the HR VLAN)
- Expected result: No open ports visible — micro-segmentation prevents the Guest VLAN from reaching HR VLAN hosts
- Document: Nmap output showing 'host unreachable' or 'filtered' for all ports

Test 6: Credential Stuffing Attack

- Using Hydra or Burp Suite, attempt a brute-force login against the Keycloak login page

- Command: hydra -l testuser -P /usr/share/wordlists/rockyou.txt http-form-post '...keycloak login form params...'
- Expected result: Account locked out after 5 failed attempts (configure Keycloak's brute force protection in Realm Settings)
- Document: Keycloak admin console showing the locked account and failed attempt logs

Test 7: Unauthorized API Access

- Attempt to call a protected API endpoint without a valid JWT bearer token
- Command: curl http://app-server/api/finance/reports (no Authorization header)
- Expected result: 401 Unauthorized response from the Nginx gateway
- Then obtain a valid token as Finance_Analyst and attempt to access an HR API endpoint — should return 403 Forbidden

6.4 Test Results Summary Template

Test Name	Attack Type	Control Tested	Result
Privilege Escalation	Insider	RBAC	**BLOCKED**
Lateral Movement	Insider	Micro-segmentation	**BLOCKED**
Data Exfiltration	Insider	DLP + Logging	**ALERTED**
MFA Bypass	Insider/External	MFA	**BLOCKED**
Port Scanning	External	Micro-segmentation	**BLOCKED**
Credential Stuffing	External	Brute Force Protection	**BLOCKED**
Unauthorized API Access	External	JWT + RBAC	**BLOCKED**

Step 7: Document Results and Suggest Improvements

7.1 Summary of Project Results

The Zero Trust Architecture implemented in this project successfully demonstrated that the core security controls — MFA, RBAC, network micro-segmentation, and centralized IAM — can prevent or detect the majority of common enterprise threats when properly configured and layered.

20. All seven security tests produced the expected results: six threats were blocked at the appropriate control layer, and one (data exfiltration) generated a detectable alert.
21. The Keycloak IAM server successfully served as the centralized Policy Decision Point, handling authentication, MFA enforcement, and RBAC policy evaluation.
22. Network micro-segmentation proved to be highly effective at preventing lateral movement — an attacker who compromised one segment had no visibility into or connectivity to other segments.
23. The combination of MFA and RBAC eliminated the most common attack path: stolen credentials alone were not sufficient for unauthorized access.

7.2 Identified Gaps and Areas for Improvement

Gap 1: No Continuous Behavioral Monitoring

The current implementation validates identity at login but does not continuously monitor user behavior during an active session. A legitimate user who is later compromised (e.g., through a session hijack) would not be detected until their next login attempt.

- Improvement: Deploy a User and Entity Behavior Analytics (UEBA) solution such as Splunk UBA or Microsoft Sentinel
- UEBA baselines normal user behavior (login time, data access volume, locations) and alerts on deviations in real time

Gap 2: No Device Posture Assessment

The simulation verified user identity and role but did not verify the security posture of the device being used. A user logging in from a malware-infected personal laptop would still pass the MFA and RBAC checks.

- Improvement: Implement a Mobile Device Management (MDM) solution such as Microsoft Intune or Jamf
- Configure conditional access policies: device must have disk encryption enabled, OS must be within one major version, antivirus must be active and updated

Gap 3: No Automated Incident Response

Currently, security events are logged and alerts are generated, but response actions (blocking an account, isolating a VM, revoking a token) require manual intervention.

- Improvement: Deploy a Security Orchestration, Automation, and Response (SOAR) platform
- Define automated playbooks: if Keycloak detects 10 failed logins in 5 minutes, automatically lock the account AND send an alert to the security team AND log the IP address in the threat intelligence feed

Gap 4: Secrets and Certificate Management

In the simulation, TLS certificates and application secrets were managed manually. In production, this creates operational risk (expired certificates, hardcoded credentials).

- Improvement: Deploy HashiCorp Vault for centralized secrets management
- Implement automatic certificate rotation using Let's Encrypt (for external services) or an internal PKI

7.3 Scaling Zero Trust to a Real Enterprise

Improvement Area	Student Lab Tool	Enterprise Production Tool
IAM / SSO	Keycloak (open source)	Okta, Microsoft Entra ID (Azure AD), Ping Identity
Network Segmentation	VirtualBox Host-Only Networks	Cisco SD-WAN, VMware NSX, AWS VPC with Security Groups
Endpoint Security	Windows Defender (basic)	CrowdStrike Falcon, Microsoft Defender for Endpoint
Security Monitoring	Grafana + Prometheus	Splunk, Microsoft Sentinel, IBM QRadar (SIEM)
ZTNA (Remote Access)	OpenVPN (basic simulation)	Zscaler Private Access, Cloudflare Access, Palo Alto Prisma
Secrets Management	Environment variables (basic)	HashiCorp Vault, AWS Secrets Manager, Azure Key Vault

7.4 Conclusion

This project demonstrated that Zero Trust Architecture is not a single product or technology but a comprehensive security philosophy implemented through a coordinated set of controls. The seven-step implementation — from understanding the principles through designing, simulating, testing, and documenting — validates that a Zero Trust approach can dramatically reduce enterprise attack surface even when adversaries already have valid credentials.

The key insight is that Zero Trust succeeds because it eliminates implicit trust at every layer. Even if an attacker steals a password, they still face MFA. Even if they bypass MFA, they still face RBAC. Even if they obtain the correct role, they still face network micro-segmentation. This defense-in-depth approach, applied consistently across identity, network, application, and data layers, represents the current state-of-the-art in enterprise security architecture.

Future work should focus on extending this model with continuous behavioral monitoring, device posture assessment, automated incident response, and formal compliance mapping to frameworks such as ISO 27001, SOC 2, and the NIST Zero Trust Architecture (SP 800-207) guidelines.